TOSHKENT SHAHRIDAGI INHA UNIVERSITETI
INHA UNIVERSITY IN TASHKENT

STUDENT NAME:

STUDENT ID NUMBER: SECTION NUMBER:

*** FINAL-TERM EXAMINATION SPRING-2023 ***

COURSE NAME:	**OBJECT ORIENTED PROGRAMMING-2**
COURSE NUMBER:	**MSC1052**
EXAMINATION DATE:	TIME:
EXAMINATION DURATION:	**60 minutes**
ADDITIONAL MATERIALS ALLOWED TO USE:	NO
SPECIAL INSTRUCTIONS:	• Maximum Marks: **25** • **FILL ANSWERS IN THE PROVIDED TABLES**

Please do not open the examination paper until directed to do so.

READ INSTRUCTIONS FIRST:

All Calculations/steps must be shown at the space provided after the question (penalty may be applicable as instructed in class)

Ambiguous answers will be awarded zero.

Desks should be free from all unnecessary items (books, notes, technology, food, water, clothes)

Use of any electronic device (Phone, iPod, iPad, laptop) is not allowed during the examination.

Cheating, talking to fellow students, singing, turning back are not allowed

Write your ID number in each page of your examination paper.

Final answers must be written by **only blue or black, non-erasable pen**. Do not use highlighters or correction pen.

STUDENT ID NUMBER:

Section A (5 Marks)

Write the correct answer in the table given below. **[Total Marks 10*0.5=5]**

Q #	1	2	3	4	5	6	7	8	9	10
Answer	T	F	F	T	F	T	T	F	T	T

State TRUE or FALSE

1. A member function can be declared static, if it does not access any non-static class members.
2. When deriving from a Base class with public Inheritance, protected members of the base class become public members of the derived class.
3. A pointer to the derived class can point to an object to the Base class.
4. A pure virtual function in a class will make that class an abstract class.
5. It is an error to have a function with exactly same signature in both the super class and its sub class.
6. Derrived classes of an abstract class that do not provide an implementation of a pure virtual function are also abstract.
7. The statement,

   Ofstream Outfile;

   Outfile.write((char*)&obj,sizeof(obj))

   Writes only data form **obj** to **outfile**.
8. The data written to a file with **write()** function can be read with **get()** function.
9. The **ios::ate** mode allows us to write data anywhere in the file.
10. Templates create different versions of a function at run time.

STUDENT ID NUMBER:

Section B (Marks =10)

Write the correct answer(A,B,C,D) in the table given below.

Q #	1	2	3	4	5	6	7	8	9	10
Answer	D	D	A,B	A	B	A	B	C	C	A,B,C
Q #	11	12	13	14	15	16	17	18		
Answer	B,D,E,F	C	C	B	D	C	A,C	A,E		

1. Which of the following features must be supported by any programming language to become a pure object-oriented programming language?

A) Encapsulation B) Inheritance C) Polymorphism D) All of These

2. The features of the base class are said to be inherited by the:

A) Constructor B) Protected class C) private class D) derived class

3. Data members which will be inherited will have to be declared as:

A) protected B) public C) private D) Both B and C

4. Virtual functions are primarily used in which of the following:

A) Inheritance B) Operator Overloading C) Encapsulation D) Data Binding

5. Which Inheritance allows to create a derived class that inherits properties from more than one base class.

A) Multilevel B) Multiple C) Hybrid D) Hierarchical

6. The current reading position, which is the index of the next byte that will be read from the file is called as:

A) Get pointer B) Set pointer C) Curr pointer D) Put pointer

STUDENT ID NUMBER:

7. If a function is declared virtual in its base class, you can still access it directly using the —-

A)Virtual Keyword B) Scope Resolution Operator C) Indirection Operator D) Address Operator

8. Which one of the following cannot be used with the virtual keyword?

A) Destructor B) Friend Function C) Constructor D) None of these

9. Which one of the following statements correctly refers to the Delete and Delete[] in C++ programming language?

A) The "Delete" is used for deleting the standard objects, while on the other hand, the "Delete[]" is used to delete the pointer objects
B) The "Delete" is a type of keyword, whereas the "Delete[]" is a type of identifier.
C) The "Delete" is used for deleting a single standard object, whereas the "Delete[]" is used for deleting an array of the multiple objects
D) Delete is syntactically correct although, if the Delete[] is used, it will obtain an error.

10. Which of the following is not correct for virtual function in C++ ?

A) Must be declared in public section of class.
B) Virtual Function can be static.
C) Virtual functions cannot be accessed using pointers.
D) Virtual Functions is defined in derrived class.

11. Given the declaration:

```
int x;
int *p;
int *q;
```

which of the following statements is/are valid:

A) p=q B) *p=56 C) p=x
D)*p=*q E) q=&x F) *p=*q

12. A static function
    A) Should be called when an object is destroyed
    B) Is closely connected with an individual object of the class
    C) Can be called using class name and function name
    D) Is used when a dummy object must be created

13. The statement **file.write((char*)&ob,sizeof(ob)):**
    A) Writes the member function of on to the file.
    B) Write the data in ob to file.
    C) Writes the member function and data of ob to file.
    D) Writes the address of ob to file.

14. How many argument A copy constructor takes

A) Zero B) One C) Two D) Arbitrary

15. If a Base class destructor is not virtual then :

A) It cannot have a function body.
B) It cannot be called.
C) It cannot be called when accessed from pointer
D) Destructor in derived class cannot be called when accessed through a pointer to the base class.

16. Find the error in the code:

```
class example
{private:
        int data;
public:
        example();
        void display()const; //Line 1
};
example::example() //Line 2
{       data = 0;}
example::display()// Line 3
{
        cout << data << endl;
}
int main()
{       example d
        d.display();
        return 0;
}
```

A) Line 1 B) Line 2 C) Line 3 D) All of these

17. Find the error in the code: **[1 Mark]**

```
class X
{private:
        int x1;
protected:
        int x2;
public:
        int x3;
};
class Y :public X
{public:
        void f()
        {       int y1, y2, y3;
                y1 = x1;//Line 1
                y2 = x2;
```

```
                y3 = x3;        }
};
class Z : X
{public:
        void f()
        {
                int z1, z2, z3;
                z1 = x1; //Line 2
                z2 = x2;
        z3 = x3;//Line 4
                }
};
int main()
{       int  p;
        Y y;
        p = y.x3; //Line 3
                        return 0;
}
```

A) Line 1 B) Line 3 C) Line 2 D) Line 4

18. Identify which of the following function template definitions is/are illegal: **[1 Mark]**

A) template<class A,B>
void fun(A,B){……}
B) template<class A, class B>
void fun(A,A){……}
C) template<class A>
void fun(A,A){……}
D) template<class T, typename R>
T fun(T,R){……}
E) template<class A>
A fun(int *A){……}

STUDENT ID NUMBER:

Section C [Marks 10*1=10]

Write the correct answer in the table given below.

Note: Assume that all necessary header files are already included.

Marking Scheme for Section C

[for correct answer(+1) for incorrect answer(- 0.5) for no answer(0)]

Q #	1	2	3	4	5	6	7	8	9	10
Answer	C	B	C	C	C	D	C	B	B	Class A=4 Class A=6 Class B y:5

1.
```
class Test
{
public:
        int i;
        void get();
};
void Test::get()
{
        std::cout << "Enter the value of i: ";
        std::cin >> i;
}
Test t;  // Global object
int main()
{
        Test t;  // local object
        t.get();
std::cout << "value of i in local t: " << t.i << 'n';       ::t.get();
std::cout << "value of i in global t: " << ::t.i << 'n';

        return 0;
}
```

A) Compile Error: Cannot have two objects with same class and name
B) Compile error in line :: t.get();
C) **Compile and runs fine**

2.
```
class Spring{
    mutable int s;
public:
Spring()
{           cout << "Inside Default constructor   " ;      }

Spring(const Spring& a)
{           cout << "Inside Copy Constructor    ";        }

};
int main()
{
        Spring obj;
        Spring a1 = obj;
        Spring a2(obj);
        return 0;
}
```

A) Inside Default constructor Inside Copy Constructor
B) Inside Default constructor Inside Copy Constructor Inside Copy Constructor
C) Inside Default constructor Inside Default constructor Inside Copy Constructor
D) Inside Copy Constructor Inside Default constructor Inside Copy Constructor

3.
```
class INHA {
private:
        int i, j;
public:
        INHA(int _i = 0, int _j = 0) : i(_i), j(_j) { }
};
class SOCIE : public INHA {
public:
        void Display(){
                cout << " i = " << i << "  j = " << j;
        }
};
int main() {
        SOCIE A;
        A.Display();
                return 0;
}
```

A) I=0

B) J=0
C) Compile Error

4. class Teacher

```
{public:
        void Define()
{               cout << "I am a Teacher"<<endl;       }
};
class Student : public Teacher
{private:
        void Define()
{               cout << "I'm a Student"<<endl;        }
};

int main()
{
        Teacher *T = new Teacher();
        Student S;
        T = &S;
        T->Define();
                return 0;
}
```

A) Garbage Value
B) Segmnetation fault
C) I am a Teacher
D) I am a Student

5. class BASE {

```
public:
        void print()  { cout << " Inside BASE"; }
};

class DERRIVED : public BASE {
public:
        void print() { cout << " Inside DERRIVED"; }
};

class CHILD : public DERRIVED{ };

int main(void)
{       CHILD C;
        C.print();
        return 0;
}
```

A) Inside BASE

B) Inside DERRIVED
C) Compile Error: ambiguous call to print()

6.
```
class Final
{private:
        int a;
        int b;
public:
        Final(int a = 0, int b = 0) { this->a = a; this->b = b; }
        static void Final1() { cout << "Inside fun1()"; }
        static void Final2() { cout << "Inside fun2()"; this->Final1(); }
};
int main()
{       Final obj;
        obj.Final2();
        return 0;
}
```

A) **Inside Final2() Inside Final1()**
B) **Inside Final2()**
C) **Compile Error: cannot call static Function**
D) **Compile Error: this cannot be used inside Final2()**

7.
```
class OOP2
{
private:
        int x;
public:
        OOP2(int x = 0) { this->x = x; }
        void change(OOP2 *t) { this = t; }
        void print() { cout << "x = " << x << endl; }
};

int main()
{
        OOP2 obj(5);
        OOP2 *ptr = new OOP2(10);
        obj.change(ptr);
        obj.print();
                return 0;
}
```

A) x=5
B) x=10
C) Compile error
D) **Run time error**

8. int main()

```
{       int  var = 55;
        fstream outfile("INHA.txt");
        outfile.write((char*)&x, sizeof(int));
        outfile.close();

        ifstream infile("INHA.txt");
        infile >> x;
        cout << "\n value=  " << x << endl;

        system("pause");
        return 0;
}
```

A) 55
B) Error
C) Garbage
D) **A**

9. template<class T>

```
void display(T x)
{
cout << " display Template:" << x << endl;
}
void display(float x)
{
cout << " display float:" << x << endl;
}
int main()
{
display(25);
display(35.34);
system("pause");
return 0;
}
```

A) display template 25
display template 35.34
B) display template 25
display float 35.34
C) display float 25
display float 35.34

D) **Error**

10. Write ouput of the following code in the table:

```
class classA
{public:
        virtual void print()const
        {               cout << "classA = " << x << endl;     }
        void doubleNum();
        classA(int a = 0);
private:
        int x;};
void classA::doubleNum()
{       x = 2 * x;}
classA::classA(int a)
{       x = a;}
class classB : public classA
{public:
        void print() const;
        void doubleNum();
        classB(int a = 0, int b = 0);
private:
        int y;};
void classB::print() const
{       classA::print();
        cout << "ClassB y: " << y << endl;}
void classB::doubleNum()
{        classA::doubleNum();
        y = 2 * y;}
classB::classB(int a, int b) : classA(a)
{       y = b;}
int main()
{       classA *ptrA;
        classA objectA(2);
        classB objectB(3, 5);
        ptrA = &objectA;
        ptrA->doubleNum();
        ptrA->print();
        cout << endl;
        ptrA = &objectB;
        ptrA->doubleNum();
        ptrA->print();
        cout << endl;
        system("pause");
        return 0;
```

STUDENT ID NUMBER:

```
}
```

ALL THE BEST ☺

Space for Rough Work

STUDENT ID NUMBER: